

Experiment No: 10

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <time.h>
#include <math.h>
#define size_x 320
#define size_y 384
#define smooth 3
typedef struct{
    int imagesize_x, imagesize_y; int **pixel; } image_t;
image_t allocate_image(const int imagesize_x, const int imagesize_y);
int mod(int z, int l); int med(int a[]);

int main() {
    image_t image_noise, image_filtered;
    int m, n, k, l, x, y, sum1, csx, csy, i, a[100];
    FILE *file_in, *file_out; char dummy[50] = "";
    file_in = fopen("i10.pgm", "r+");
    file_out = fopen("smooth_image_median.pgm", "w");
    fgets(dummy, 50, file_in);
    do { fgets(dummy, 50, file_in); } while(dummy[0] != '#'); fgets(dummy, 50, file_in);
    fprintf(file_out, "P2\n%d %d\n255\n", (size_x), (size_y));

    image_noise = allocate_image(size_x, size_y);
    image_filtered = allocate_image(size_x, size_y);
```

Experiment No: 10

```
for (n = 0; n < size_y; n++) {
    for (m = 0; m < size_x; m++) {
        fscanf(file_in, "%d", &image_noise.pixel[m][n]); } }

csx=smooth/2; csy=smooth/2;
for (n = 0; n < size_y; n++) {
    for (m = 0; m < size_x; m++) {
        sum1=0; i=0;
        for(k=0;k<smooth;k++) {
            for(l=0;l<smooth;l++) {
                x=m+k-csx; y=n+l-csy;
                a[i]=image_noise.pixel[mod(x,size_x)][mod(y,size_y)];
                i++;
            } }
        image_filtered.pixel[m][n]=med(a);
    } }

/* Writing image (pgm) file */
for (n = 0; n < size_y; n++) {
    for (m = 0; m < size_x; m++) {
        fprintf(file_out, "%d ", image_filtered.pixel[m][n]);
    } } fclose(file_out); }

int mod(int z, int l) {
    if( z >= 0 && z < l ) return z;
    else if( z < 0 ) return (z+l);
    else if( z > (l-1)) return (z-l); }

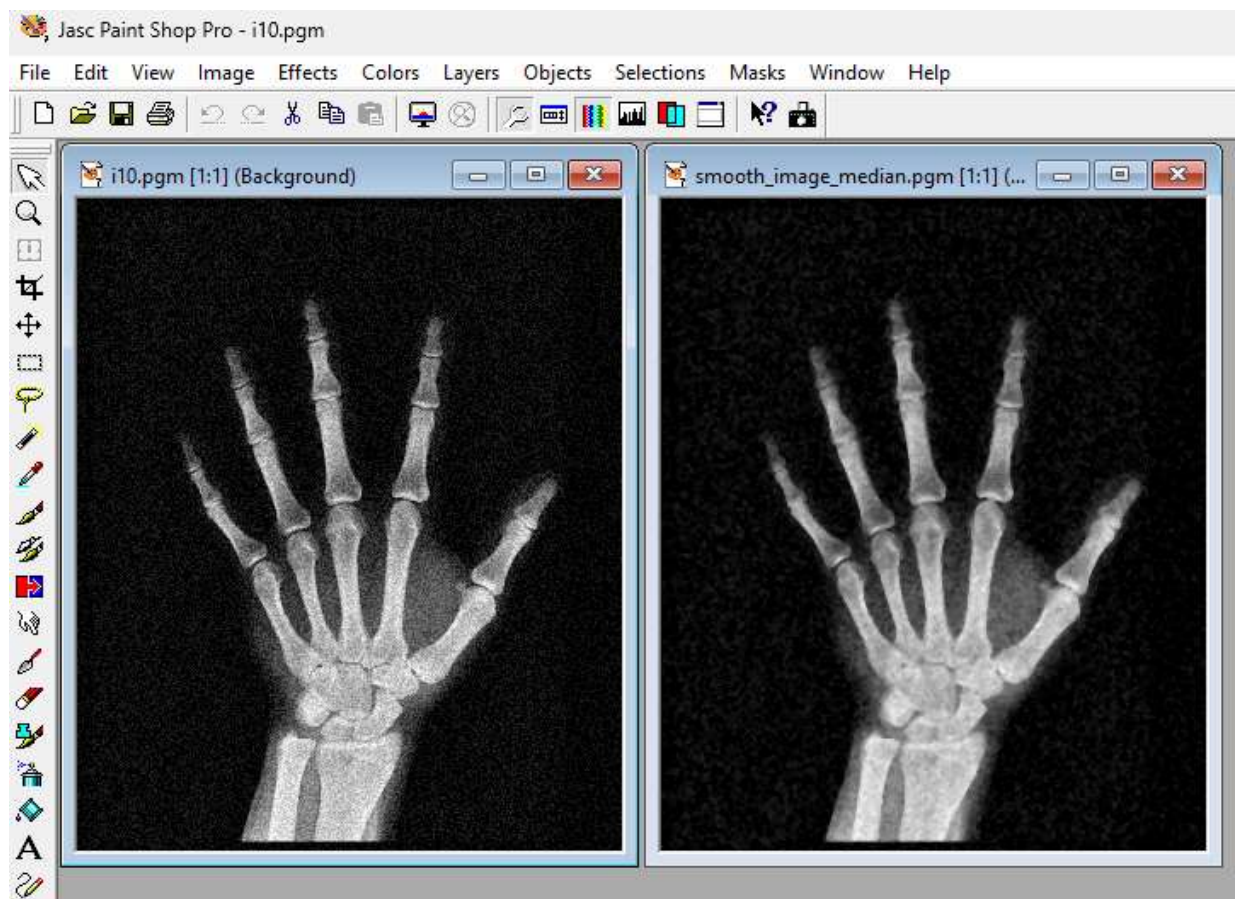
int med(int a[]) {
    int temp,i,j;
    for(j=1;j<9;j++)
        for(i=0;i<9-j;i++)
            if(a[i]>a[i+1])
            { temp=a[i];
              a[i]=a[i+1];
              a[i+1]=temp; } return a[4]; }

image_t allocate_image(const int imagesize_x, const int imagesize_y) {
    image_t result;
    int x = 0, y = 0;

    result.imagesize_x = imagesize_x;
    result.imagesize_y = imagesize_y;

    result.pixel =(int **) calloc(imagesize_x, sizeof(int*));

    for(x = 0; x < imagesize_x; x++) {
        result.pixel[x] =(int*) calloc(imagesize_y, sizeof(int));
        for(y = 0; y < imagesize_y; y++)
            { result.pixel[x][y] = 0; }
    } return result;
}
```



Left-side-Figure 10(a): Noisy image (i10.pgm)

Right-side-Figure 10(b): Noisy removed median image (smooth_image_median.pgm)